

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Вятский государственный университет»
(ВятГУ)

ОТЧЕТ
ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ,
ТЕХНОЛОГИЧЕСКОЙ (ПРОЕКТНО-ТЕХНОЛОГИЧЕСКОЙ) ПРАКТИКЕ

Зуа Анжело Абреу
(Ф.И.О. обучающегося)

02.03.02.51. Фундаментальная информатика и информационные технологии.
Разработка программного обеспечения

(направление подготовки (специальность), направленность (профиль))

Место прохождения практики ООО «Автоматизация Вашего Предприятия»
(наименование организации, структурного подразделения организации)

(наименование организации, структурного подразделения организации)

Итоговая оценка:

Руководитель

практики от университета

27.12.2025

(дата)

(подпись)

Фищева И.Н.

(Ф.И.О.)

Киров, 2025 г.

СОДЕРЖАНИЕ

Введение.....	3
Отчет о выполнении индивидуального задания.....	5
Постановка задания.....	5
Обзор имеющихся подходов к решению.....	6
Обоснование выбора методов.....	6
Описание решения.....	7
Заключение.....	30
Библиографический список.....	33
Приложение.....	34

Введение

Производственная практика, технологическая (проектно-технологическая) практика проходила в ООО «Автоматизация Вашего Предприятия» с 06.10.2025 г. по 28.12.2025 г.

Целью производственной практики является закрепление и углубление студентами полученных теоретических знаний, приобретение ими практических навыков и компетенций, подготовка к профессиональной деятельности, в основном путём самостоятельного решения задач, сформулированных в рамках индивидуального задания.

Задачи производственной практики:

- профессиональная ориентация студентов, формирование полного представления о своей профессии;

- решение прикладных задач в области применения современных математических методов и информационных технологий;

- получение практических навыков и компетенций в организации работы в области применения современных математических методов и информационных технологий

- приобретение студентами навыков выполнения работ по выбранному направлению в рамках реального производственного или учебного процесса на базе конкретного предприятия, организации или учреждения, включая учебные заведения;

- получение навыков научно-исследовательской деятельности;

- получение опыта применения математического аппарата и современных информационных технологий для анализа и решения конкретных производственных задач

- развитие и закрепление практических навыков по применению пакетов прикладных программ, разработке прикладного и системного программного обеспечения;

- моделирование производственных, финансовых, управленческих процессов;

- получение навыков самостоятельного или коллективного решения конкретных профессиональных задач;

- получение практических навыков в области организации и управления при проведении исследований;

- анализ возможности внедрения результатов исследований в производственный процесс.

Индивидуальное задание на период прохождения производственной практики заключалась в **интеграции системы самовывоза в кроссплатформенное мобильное приложение, включая функционал онлайн-оплаты**. Проект охватывал все этапы: от архитектурного проектирования до внедрения и тестирования, с целью создания комплексного, масштабируемого решения, ориентированного на пользовательский опыт (UX).

Отчет о выполнении индивидуального задания

Постановка задания

Разработать систему самовывоза для ресторана, интегрированную с онлайн-платежами, которая позволяет пользователям оформлять заказы через мобильное приложение и забирать их в удобное время в выбранной точке выдачи. Система должна обеспечивать безопасную обработку платежей, уведомления в реальном времени и удобный пользовательский интерфейс.

Цель, задачи, которые необходимо решить для достижения поставленной цели

Цель:

Создать модульное, слабосвязанное мобильное приложение с бэкенд-API для автоматизации процесса самовывоза заказов в ресторане.

Задачи:

1. аналитические:
 - исследовать современные подходы к интеграции систем самовывоза;
 - проанализировать платёжные шлюзы и выбрать оптимальный для проекта;
 - изучить требования к UX для повышения конверсии.
2. архитектурные:
 - спроектировать модульную архитектуру приложения;
 - определить стек технологий для фронтенда и бэкенда;
 - разработать схему взаимодействия компонентов системы.
3. технической реализации:
 - разработать бэкенд-API на NestJS/TypeScript;
 - настроить базы данных (MySQL, Redis, GraphQL);

- интегрировать платёжный шлюз (YooKassa);
 - реализовать кроссплатформенный клиент на Flutter/Dart;
 - реализовать real-time уведомления через WebSocket;
4. тестирования и оптимизации:
- протестировать функциональность модуля;
 - оптимизировать производительность критических операций;
 - провести UX-тестирование пользовательских сценариев.
5. обзор имеющихся подходов к решению:
- преимущества для потребителя;
 - экономия времени (отсутствие очередей, бесплатная доставка);
 - гибкость выбора времени получения;
 - большой контроль над процессом покупки;
 - преимущества для компании;
 - снижение логистических затрат;
 - увеличение трафика в физических точках (импульсивные покупки);
 - сбор данных о поведении потребителей.

Таблица 1 - Архитектурные подходы

Подход	Описание
Микросервисная архитектура	Подходит для крупных проектов, обеспечивает масштабируемость
Модульный монолит	Лучше для средних проектов, проще внедрение и развертывание

- технические компоненты системы;
- каталог и инвентаризация (реальное время);
- выбор точки выдачи;

- выбор времени получения;
- обработка заказа;
- система уведомлений;

Обоснование выбора методов

Архитектурное решение

Выбрана архитектура «клиент-сервер» с модульным монолитным API, так как:

- обеспечивает простоту внедрения и развертывания;
- не уступает в организации кода микросервисам;
- лучше подходит для масштаба данного проекта.

Таблица 2 - Паттерны проектирования

Паттерн	Где	Обоснование
Repository/Module	Бэкенд	Абстрагирование логики доступа к данным
BLoC	Фронтенд (Flutter)	Четкое отделение бизнес-логики от UI

Таблица 3 - Выбор технологического стека

Компонент	Технология	Обоснование
Фронтенд	Flutter (Dart)	Кроссплатформенность (iOS/Android), высокая производительность
Бэкенд	NestJS (TypeScript)	Модульный, объектно-ориентированный, вдохновлен Angular
API	RESTful +	Стандарт индустрии, простой и

	GraphQL	эффективный
Платежи		
БД	MySQL 8.0	Зрелая, надежная, ACID-транзакции
ORM	Prisma	TypeScript-безопасность, управляемые миграции
Real-time	WebSocket	Двунаправленная связь
Кэш/Сессии	Redis	Высокопроизводительная, in-memory

Таблица 4 - Выбор платежного шлюза Юкасса

Критерий	Обоснование
Стоимость	Конкурентоспособные комиссии
Способы оплаты	Карты, цифровые кошельки, банковские переводы
Безопасность	Соответствие PCI DSS
UX	Оплата без перенаправления (платежные токены)

Описание решения

Системы самовывоза для ресторана

Бизнес-модель (или «Купи и заberi») зарекомендовала себя как один из фундаментальных столпов современной электронной коммерции. Она служит мостом между удобством онлайн-покупок и оперативностью физического получения товара клиентом. Согласно бизнес-литературе, эта модель дает преимущества как потребителям, так и компаниям.

преимущества для потребителя:

- экономия времени благодаря отсутствию необходимости стоять в очередях и оплате доставки;

- гибкость выбора времени получения заказа;
- большой контроль над процессом покупки.
- преимущества для компании:
 - снижение логистических затрат и расходов на достав;
 - увеличение трафика в физических точках продаж (импульсивные покупки);
 - сбор ценных данных о поведении потребителей.
- техническая реализация системы самообслуживания требует сложной интеграции между онлайн-средой и операционной деятельностью.

Основные компоненты включают:

- каталог и инвентаризация: Централизованная система, отражающая наличие товаров в реальном времени для каждой точки выдачи;
- выбор точки выдачи: Интерфейс, позволяющий пользователю выбирать между различными локациями на основе местоположения или предпочтений;
- выбор времени получения: Модуль, управляющий пропускной способностью обслуживания по временным интервалам;
- обработка заказа: Серверная часть (бэкенд), которая объединяет информацию о платеже, товарах, историю заказов, детали получения и т.д;
- уведомления: Система связи для информирования клиента о статусе заказа;

Интеграция онлайн-платежей

Является одним из наиболее критически важных компонентов любой системы электронной коммерции. Она включает в себя безопасное соединение

между приложением и платежным шлюзом (Gateway), который выступает посредником между продавцом, клиентом и финансовыми учреждениями.

Для данного проекта выбранным платежным шлюзом стал ЮKassa.

Современные платежные шлюзы предлагают SDK и API, которые берут на себя большую часть сложности обработки транзакций.

Выбор шлюза включает анализ следующих факторов

Таблица 5 - анализ выбора юкасса

Критерий	Описание
Стоимость (Затраты)	Комиссия за транзакции, абонентская плата и стоимость внедрения
Способы оплаты	Поддержка кредитных и дебетовых карт, цифровых кошельков и банковских переводов
Безопасность	Соответствие стандарту PCI DSS для безопасной обработки данных
Пользовательский опыт	Удобный процесс оформления заказа с возможностью оплаты без перенаправления на сторонние страницы

Для проекта платежный процесс был спроектирован таким образом, чтобы минимизировать трение (сложности для пользователя). Для этого использовались платежные токены, что позволило избежать обработки или хранения чувствительных данных напрямую в бэкенде приложения.

Введение Архитектура современных мобильных приложений

Эволюция разработки программного обеспечения привела к предпочтению модульных и слабосвязанных архитектур. В контексте мобильных приложений распространенными практиками являются архитектура микросервисов на серверной стороне (бэкенд) и компонентно-ориентированная архитектура на клиентской стороне (фронтенд).

Задачи исследования

Архитектурные решения

После анализа требований была выбрана архитектура «клиент-сервер» с модульным монолитным API. Хотя архитектура микросервисов интересна для крупных проектов, хорошо структурированный монолитный подход лучше подходит для масштаба данного проекта, обеспечивая простоту внедрения и развертывания без ущерба для организации кода.

Архитектура состоит из трех основных уровней:

- Уровень представления (фронтенд): Мобильное приложение на Flutter.
- Уровень бизнес-логики (бэкенд): API, разработанное на NestJS.
- Уровень данных: Реляционная база данных (MySQL) и база данных в памяти (Redis).

Паттерн Repository использовался на бэкенде для абстрагирования логики доступа к данным. На фронтенде был применен паттерн BLoC (Business Logic Component) для управления состоянием приложения, что позволило четко разделить бизнес-логику от пользовательского интерфейса

Экспериментальное исследование качества реализованного решения или тестирование полученного программного продукта

Диаграмма архитектуры и описание компонентов

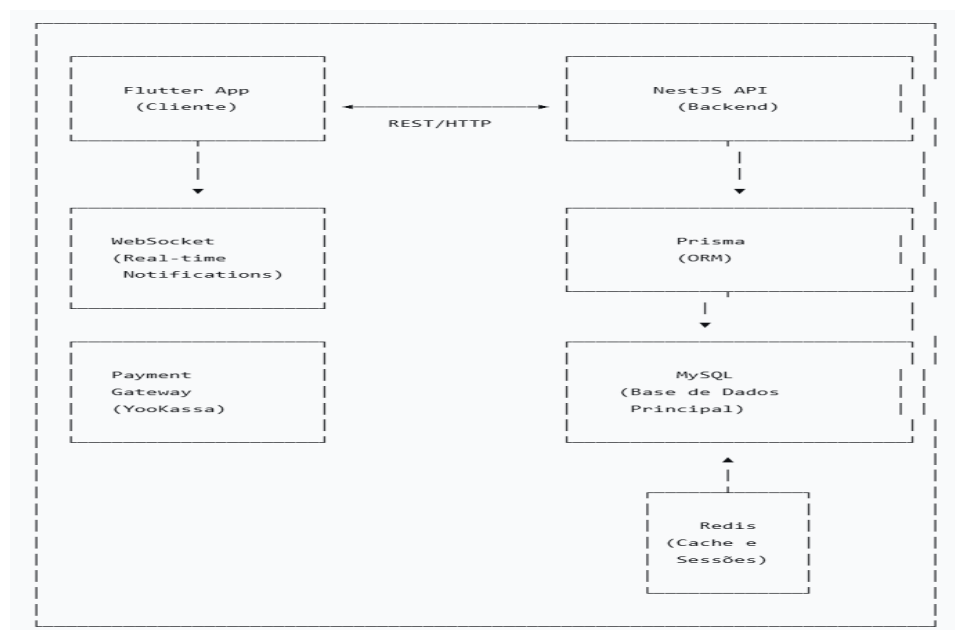


Рисунок 1 - Пример архитектурного решения

Flutter App (мобильный клиент)

Приложение Flutter представляет собой уровень представления системы и является точкой прямого взаимодействия с конечным пользователем. Разработанное на Dart, это приложение отвечает за:

- пользовательский интерфейс (UI): Отрисовку всех экранов приложения;
- управление состоянием: Использование паттерна BLoC для реактивного управления состоянием;
- коммуникацию с бэкендом: HTTPS/GraphQL - запросы к API на NestJS с использованием библиотеки dio;
- websocket-соединение: Получение уведомлений в реальном времени;
- интеграцию с платежным шлюзом: Использование SDK YooKassa;
- поток данных в клиенте:
 - пользователь взаимодействует с интерфейсом;
 - событие отправляется соответствующему BloC-компоненту;
 - bloc обрабатывает логику и выполняет вызовы API;
 - получается ответ и обновляется состояние;
 - интерфейс реагирует на изменения состояния.

NestJS API (бэкенд)

Бэкенд, разработанный на NestJS, является сердцем системы. Он отвечает за всю бизнес-логику, обработку данных и оркестрацию остальных компонентов.

Таблица 6 - Основные реализованные модули

Модуль	Обязанности
PickupPointsModule	Управление точками выдачи, временем работы, доступностью и кэшированием

OrdersModule	Создание, просмотр, обновление и управление жизненным циклом заказов
PaymentsModule	Интеграция с YooKassa, обработка вебхуков
NotificationsModule	Управление WebSocket и отправка уведомлений

MySQL (основная база данных)

был выбран в качестве основной реляционной базы данных благодаря своей надежности и поддержке ACID-транзакций.

Таблица 7 - Основные таблица MySQL

Таблица	Описание
users	Данные зарегистрированных пользователей
pickup_points	Информация о точках выдачи
working_hours	Часы работы
orders	Записи о выполненных заказах
order_items	Товары, входящие в каждый заказ
payments	Информация о транзакциях оплаты

Основные реализованные индексы:

- idx_orders_user_id - быстрые запросы заказов по пользователю;
- idx_orders_pickup_time - проверка доступности временных интервалов;
- idx_pickup_points_coordinates - географический поиск.

Prisma (ORM)

Prisma выступает в качестве слоя абстракции между приложением NestJS и MySQL, обеспечивая:

- type-Safety (безопасность типов): Гарантию корректности запросов на этапе компиляции;

- управляемые миграции: Контроль версий схемы базы данных;
- упрощенные связи: Интуитивно понятную работу со связями между таблицами.

Redis (кэш и сессии)

Redis был внедрен для двух основных целей:

Таблица 8 - Основные реализованные Redis

Тип данных	TTL (время жизни)	Стратегия инвалидации
Список точек выдачи	1 час	Инвалидация по времени (TTL)
Доступные временные интервалы	5 минут	Инвалидация после создания заказа
Сессия пользователя	24 часа	Инвалидация по времени и при выходе из системы

управление сессиями:

- горизонтальная масштабируемость (несколько экземпляров приложения разделяют общее состояние);
- удаленный отзыв сессий;
- быстрое время ответа при проверках аутентификации.

WebSocket (уведомления в реальном времени)

WebSocket устанавливает двунаправленный канал связи между сервером и клиентом.

поток уведомлений:

- пользователь аутентифицируется и устанавливает WebSocket-соединение;

- сервер связывает идентификатор пользователя с сокетом;
- когда статус заказа изменяется, сервер публикует сообщение;
- клиент получает сообщение и обновляет интерфейс.

типы уведомлений:

- order_confirmed – подтверждение создания заказа;
- payment_confirmed – оплата подтверждена;
- order_ready – заказ готов к выдаче;
- order_reminder – напоминание о приближении времени получения.

Платежный шлюз (YooKassa)

YooKassa был выбран благодаря своей популярности на российском рынке, обширной документации и поддержке SDK.

Безопасный платежный поток

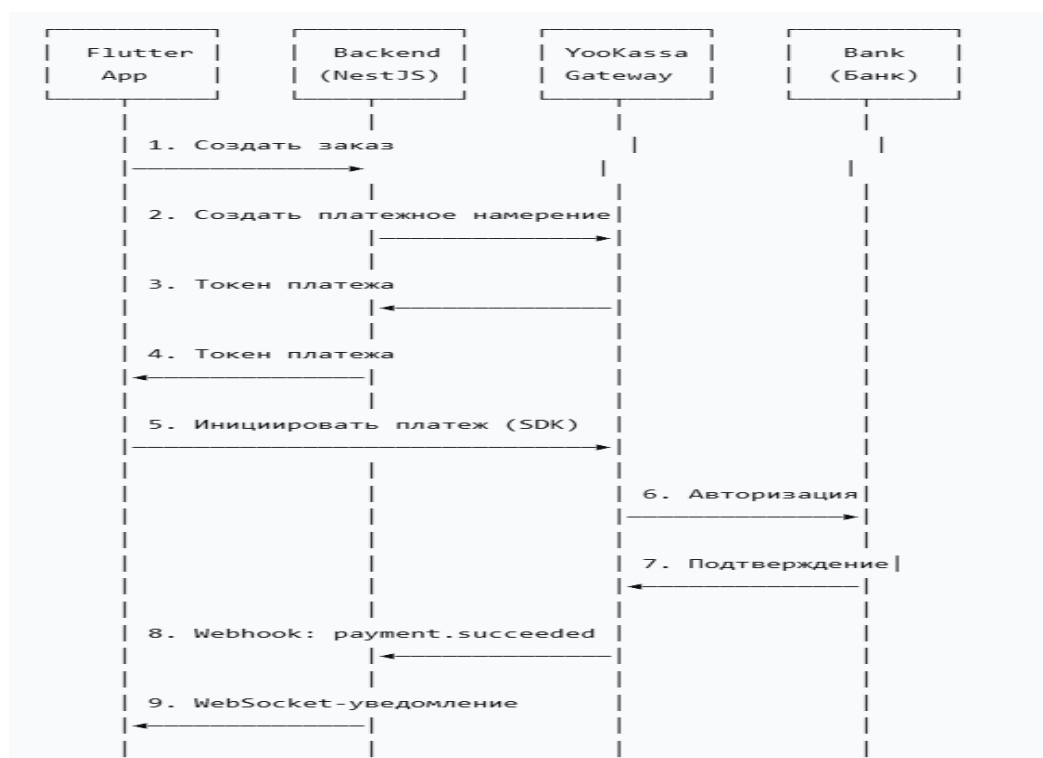


Рисунок 2 - пример архитектурного решения реализации Юкасса

Настройка среды разработки

Настройка бэкенда (NestJS):

шаг 1: установка и начальная настройка:

- установка Node.js и npm: Проверка и установка LTS-версии Node.js (v20.x);
- установка NestJS CLI: **`npm install -g @nestjs/cli`**;
- создание проекта: **`nest new backend`**;
- структурирование каталогов: **`mkdir -p src/modules src/common src/config`**;
- настройка TypeScript: Корректировка `tsconfig.json` для использования абсолютных путей;
- настройка переменных окружения: Создание модуля конфигурации с помощью `@nestjs/config`;

Настройка фронтенда (Flutter)

шаг 2: настройка Flutter:

- установка Flutter SDK: Версия 3.16.x;
- создание проекта: **`flutter create pickup_app --org com.company`**;
- настройка зависимостей: Добавление в `pubspec.yaml`;

```
dependencies:  
  flutter_bloc: ^8.1.3  
  dio: ^5.3.3  
  web_socket_channel: ^2.4.0  
  yookassa_sdk_flutter: ^1.0.0  
  equatable: ^2.0.5  
  get_it: ^7.6.4  
  go_router: ^12.0.0  
  flutter_secure_storage: ^9.0.0
```

Рисунок 3 - Зависимости, необходимые для Flutter

- настройка ресурсов (Assets): Директория **assets/** для изображений, иконок и шрифтов.

Настройка инфраструктуры и базы данных

Настройка базы данных MySQL:

шаг 3: docker Compose для среды разработки;

```
version: '3.8'
services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: pickup_app
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

volumes:
  mysql_data:
```

Рисунок 4 - Пример настроек в файле Docker

шаг 4: настройка Prisma в NestJS:

- установка: ***npm install -D prisma u npx prisma init;***
- определение схемы: Создание ***prisma/schema.prisma;***
- выполнение миграций:

```
npx prisma migrate dev --name init
npx prisma generate
```

- настройка Redis для кэширования и сессий

шаг 5: установка и настройка:

- установка модулей;

```
npm install @nestjs/bull bull ioredis
```

- настройка модуля кэширования;

```
@Module({
  imports: [
    CacheModule.registerAsync({
      isGlobal: true,
      useFactory: async (configService: ConfigService) => ({
        store: redisStore,
        host: configService.get('redis.host'),
        port: configService.get('redis.port'),
        ttl: 60 * 5,
      }),
      inject: [ConfigService],
    }),
  ],
})
export class CacheModule {}
```

Рисунок 5 - Пример настроек кеша на бэкенде

- разработка бэкенда - реализация по слоям;
- реализация модуля точек выдачи.

шаг 6: создание базовой структуры:

```
nest generate module modules/pickup-points
nest generate service modules/pickup-points
nest generate controller modules/pickup-points
```

шаг 7: определение DTO

```
export class PickupPointResponseDto {
  id: string;
  name: string;
  address: string;
  city: string;
  latitude: number;
  longitude: number;
  workingHours: WorkingHoursDto[];
}
```

Рисунок 6 - Класса DTO PickupPoint на бэкенде

шаг 8: реализация Repository:

```
@Injectable()
export class PickupPointsRepository {
  constructor(private prisma: PrismaService) {}

  async findAll(): Promise<PickupPoint[]> {
    return this.prisma.pickupPoint.findMany({
      include: { workingHours: true },
    });
  }

  async findAvailableSlots(pickupPointId: string, date: Date): Promise<TimeSlot[]> {
    const existingOrders = await this.prisma.order.findMany({
      where: {
        pickupPointId,
        pickupTime: {
          gte: new Date(date.setHours(0, 0, 0, 0)),
          lt: new Date(date.setHours(23, 59, 59, 999)),
        },
      },
    });
    // Лógica de cálculo de disponibilidade
  }
}
```

Рисунок 7 - Класса реализация repository

шаг 9: реализация Service с кэшированием:

```
@Injectable()
export class PickupPointsService {
  async findAll(): Promise<PickupPointResponseDto[]> {
    const cacheKey = 'pickup-points:all';
    const cached = await this.cacheManager.get(cacheKey);

    if (cached) return cached;

    const points = await this.repository.findAll();
    const response = points.map(point => this.toResponseDto(point));

    await this.cacheManager.set(cacheKey, response, 3600);
    return response;
  }
}
```

Рисунок 8 - Класса реализация service

- реализация модуля заказов

шаг 10: определение статуса заказа:

```
export enum OrderStatus {  
    PENDING = 'PENDING',  
    PAYMENT_PROCESSING = 'PAYMENT_PROCESSING',  
    PAYMENT_CONFIRMED = 'PAYMENT_CONFIRMED',  
    READY_FOR_PICKUP = 'READY_FOR_PICKUP',  
    COMPLETED = 'COMPLETED',  
    CANCELLED = 'CANCELLED',  
    EXPIRED = 'EXPIRED',  
}
```

Р

исунок 9 - Enum статус заказ

шаг 11: реализация создания заказа с использованием транзакции:

```

async createOrder(createOrderDto: CreateOrderDto, userId: string): Promise<OrderResponseDto> {
  const isSlotAvailable = await this.validateSlotAvailability(
    createOrderDto.pickupPointId,
    createOrderDto.pickupTime,
  );

  if (!isSlotAvailable) {
    throw new ConflictException('Horário não disponível');
  }

  return this.prisma.$transaction(async (tx) => {
    const order = await tx.order.create({
      data: {
        userId,
        totalAmount: createOrderDto.totalAmount,
        status: OrderStatus.PENDING,
        pickupPointId: createOrderDto.pickupPointId,
        pickupTime: new Date(createOrderDto.pickupTime),
        items: { create: createOrderDto.items },
      },
    });

    const paymentIntent = await this.paymentService.createPaymentIntent({
      orderId: order.id,
      amount: order.totalAmount,
    });

    return this.toResponseDto(order, paymentIntent);
  });
}

```

Рисунок 10 - Пример реализация создания заказа с использованием транзакции

- реализация интеграции с платежным шлюзом.

шаг 12: создание сервиса Юкасса:

```

@Injectable()
export class YooKassaService {
  private client: YooKassaClient;

  constructor(private configService: ConfigService) {
    this.client = new YooKassaClient({
      shopId: this.configService.get('yookassa.shopId'),
      secretKey: this.configService.get('yookassa.secretKey'),
    });
  }

  async createPaymentIntent(params: CreatePaymentParams): Promise<PaymentIntent> {
    const payment = await this.client.createPayment({
      amount: { value: params.amount.toFixed(2), currency: 'RUB' },
      capture: true,
      confirmation: { type: 'embedded' },
      metadata: { orderId: params.orderId },
    });

    return {
      id: payment.id,
      confirmationToken: payment.confirmation.confirmation_token,
    };
  }

  async handleWebhook(webhookData: any): Promise<void> {
    if (webhookData.event === 'payment.succeeded') {
      const orderId = webhookData.object.metadata.orderId;
      await this.updateOrderPaymentStatus(orderId, OrderStatus.PAYMENT_CONFIRMED);
      await this.notificationsGateway.sendOrderStatusUpdate(orderId, OrderStatus.PAYMENT_CONFIRMED);
    }
  }
}

```

Рисунок 11 - Реализация создания заказа с использованием транзакции

- реализация WebSocket для уведомлений:

шаг 13: создание шлюза (Gateway) уведомлений:

```

@WebSocketGateway({
  cors: { origin: ['http://localhost:3000', 'http://localhost:8080'] },
  namespace: '/notifications',
})
export class NotificationsGateway implements OnGatewayConnection, OnGatewayDisconnect {
  @WebSocketServer()
  server: Server;

  private userSockets: Map<string, string[]> = new Map();

  handleConnection(client: Socket, ...args: any[]) {
    const userId = client.handshake.auth.userId;
    if (userId) {
      const existing = this.userSockets.get(userId) || [];
      this.userSockets.set(userId, [...existing, client.id]);
      client.join(`user:${userId}`);
    }
  }

  sendOrderStatusUpdate(orderId: string, status: OrderStatus) {
    this.server.emit('order:status', {
      orderId,
      status,
      timestamp: new Date().toISOString(),
    });
  }
}

```

Рисунок 12 - Пример реализация WebSocket для уведомлений

- разработка фронтенда – реализация по слоям;
- настройка внедрения зависимостей.

шаг 14: настройка get_it:


```

final sl = GetIt.instance;

Future<void> initDependencies() async {
  // Services
  sl.registerLazySingleton<ApiService>(() => ApiService(
    Dio(BaseOptions(baseUrl: Constants.apiUrl)),
  ));
  sl.registerLazySingleton<WebSocketService>(() => WebSocketService());
  sl.registerLazySingleton<PaymentService>(() => PaymentService());

  // Repositories
  sl.registerLazySingleton<OrderRepository>(() => OrderRepositoryImpl(
    apiService: sl(),
    websocketService: sl(),
  ));

  // BLoCs
  sl.registerFactory<CheckoutBloc>(() => CheckoutBloc(
    orderRepository: sl(),
    paymentService: sl(),
  ));
}

```

Рисунок 13 - Пример настройка get_it на Flutter

- реализация BLoC для оформления заказа (Checkout).

шаг 15: реализация BLoC:

```

class CheckoutBloc extends Bloc<CheckoutEvent, CheckoutState> {
  final OrderRepository orderRepository;
  final PaymentService paymentService;

  CheckoutBloc({required this.orderRepository, required this.paymentService})
    : super(CheckoutInitial()) {
    on<SelectPickupPoint>(_onSelectPickupPoint);
    on<SelectPickupTime>(_onSelectPickupTime);
    on<SubmitOrder>(_onSubmitOrder);
    on<PaymentCompleted>(_onPaymentCompleted);
  }

  Future<void> _onSubmitOrder(SubmitOrder event, Emitter<CheckoutState> emit) async {
    emit(CheckoutLoading());

    try {
      final order = await orderRepository.createOrder(
        items: event.items,
        pickupPointId: state.selectedPickupPoint!.id,
        pickupTime: state.selectedTime!,
      );

      emit(OrderCreated(order));

      final paymentResult = await paymentService.startPayment(
        orderId: order.id,
        amount: order.totalAmount,
      );

      if (paymentResult.requiresConfirmation) {
        emit(PaymentPending(order, paymentResult));
      }
    } catch (e) {
      emit(CheckoutError(e.toString()));
    }
  }
}

```

Рисунок 14 - Пример реализация BloC Flutter

- реализация экрана выбора точки выдачи.

шаг 16: экран выбора:

```
class PickupPointSelectionScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return BlocProvider(
      create: (context) => sl<PickupPointBloc>()..add(LoadPickupPoints()),
      child: Scaffold(
        appBar: AppBar(title: Text('Selezione o ponto de retirada')),
        body: BlocBuilder<PickupPointBloc, PickupPointState>(
          builder: (context, state) {
            if (state is PickupPointsLoading) {
              return Center(child: CircularProgressIndicator());
            }
            if (state is PickupPointsLoaded) {
              return ListView.builder(
                itemCount: state.pickupPoints.length,
                itemBuilder: (context, index) {
                  return PickupPointCard(
                    pickupPoint: state.pickupPoints[index],
                    onTap: () {
                      context.read<CheckoutBloc>().add(SelectPickupPoint(point));
                      Navigator.pop(context);
                    },
                  );
                },
              );
            }
            return Container();
          },
        ),
      ),
    );
  }
}
```

Рисунок 15 - Пример экран выбора Flutter

- реализация экрана оформления заказа (Checkout) с интеграцией оплаты.

шаг 17: экран оформления заказа:

```
class CheckoutScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return BlocConsumer<CheckoutBloc, CheckoutState>(
      listener: (context, state) {
        if (state is PaymentPending) {
          YookassaSdkFlutter.startPayment(
            paymentToken: state.paymentData.confirmationToken,
            onSuccess: (paymentId) {
              context.read<CheckoutBloc>().add(PaymentCompleted(
                orderId: state.order.id,
                paymentId: paymentId,
              ));
            },
            onError: (error) {
              context.read<CheckoutBloc>().add(PaymentFailed(error));
            },
          );
        }
        if (state is PaymentSuccess) {
          Navigator.pushReplacement(
            context,
            MaterialPageRoute(builder: (_) => OrderStatusScreen(orderId: state.order.id)),
          );
        }
      },
      builder: (context, state) {
        return Scaffold(
          appBar: AppBar(title: Text('Finalizar Pedido')),
          body: SingleChildScrollView(
            child: Column(
              children: [
                OrderItemsSummary(items: state.orderItems),
                PickupInfoCard(
                  pickupPoint: state.selectedPickupPoint,
                  pickupTime: state.selectedTime,
                ),
                PriceSummary(subtotal: state.subtotal, total: state.total),
                CustomButton(
                  text: 'Pagar ${formatCurrency(state.total)}',
                  onPressed: () {
                    context.read<CheckoutBloc>().add(SubmitOrder(items: state.orderItems));
                  },
                ),
              ],
            ),
          ),
        );
      },
    );
  }
}
```

Рисунок 16 - Пример реализация экрана оформления заказа

- тестирование и валидация;
- модульные тесты.

шаг 18: Тестирование бэкенда:

```

describe('OrdersService', () => {
  let service: OrdersService;

  beforeEach(async () => {
    const module = await Test.createTestingModule({
      providers: [
        OrdersService,
        { provide: OrdersRepository, useValue: createMockRepository() },
        { provide: PaymentService, useValue: createMockPaymentService() },
      ],
    }).compile();

    service = module.get<OrdersService>(OrdersService);
  });

  describe('createOrder', () => {
    it('deve criar um pedido com sucesso', async () => {
      const createOrderDto = { /* ... */ };
      const result = await service.createOrder(createOrderDto, 'user-1');

      expect(result.id).toBeDefined();
      expect(result.status).toEqual(OrderStatus.PENDING);
    });

    it('deve lançar erro quando horário não está disponível', async () => {
      jest.spyOn(repository, 'isSlotAvailable').mockResolvedValue(false);

      await expect(service.createOrder(createOrderDto, 'user-1'))
        .rejects.toThrow(ConflictException);
    });
  });
});

```

Рисунок 17 - Пример реализация тестирование бэкенда

шаг 19: Тестирование фронтенда:

```
void main() {  
  group('CheckoutBloc', () {  
    late CheckoutBloc bloc;  
  
    setUp(() {  
      bloc = CheckoutBloc(  
        orderRepository: MockOrderRepository(),  
        paymentService: MockPaymentService(),  
      );  
    });  
  
    test('estado inicial deve ser CheckoutInitial', () {  
      expect(bloc.state, equals(CheckoutInitial()));  
    });  
  
    group('SubmitOrder', () {  
      test('deve emitir OrderCreated e PaymentPending em caso de sucesso', () {  
        when(mockOrderRepository.createOrder(any, any, any))  
          .thenAnswer((_) async => order);  
  
        bloc.add(SubmitOrder(items: items));  
  
        expectLater(  
          bloc.stream,  
          emitsInOrder([CheckoutLoading(), OrderCreated(order), PaymentPending(order, paymentData)]),  
        );  
      });  
    });  
  });  
}
```

Рисунок 18 - Пример реализация тестирования фронтенда

- интеграционное и нагрузочное тестирование.

шаг 20: интеграционное тестирование API:

```
describe('Orders API (e2e)', () => {
  let app: INestApplication;

  beforeAll(async () => {
    const moduleFixture = await Test.createTestingModule({
      imports: [AppModule],
    }).compile();

    app = moduleFixture.createNestApplication();
    await app.init();
  });

  it('POST /orders - deve criar um pedido', () => {
    return request(app.getHttpServer())
      .post('/orders')
      .send(createOrderDto)
      .expect(201)
      .expect((res) => {
        expect(res.body.id).toBeDefined();
        expect(res.body.status).toBe('PENDING');
      });
  });
});
```

Рисунок 19 - Пример интеграционное тестирование API

шаг 21: нагрузочное тестирование с помощью K6:

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '30s', target: 20 },
    { duration: '1m', target: 20 },
    { duration: '30s', target: 0 },
  ],
};

export default function () {
  const res = http.get('http://localhost:3000/pickup-points');
  check(res, {
    'status is 200': (r) => r.status === 200,
    'response time < 200ms': (r) => r.timings.duration < 200,
  });
  sleep(1);
}
```

Рисунок 20 - Пример нагрузочное тестирование с помощью K6

Структура каталогов проекта бэкенда:

```
backend/
├── src/
│   ├── modules/
│   │   ├── orders/
│   │   │   ├── dto/
│   │   │   ├── entities/
│   │   │   ├── orders.controller.ts
│   │   │   ├── orders.module.ts
│   │   │   ├── orders.service.ts
│   │   │   └── orders.repository.ts
│   │   ├── payments/
│   │   │   ├── dto/
│   │   │   ├── payments.controller.ts
│   │   │   ├── payments.module.ts
│   │   │   ├── payments.service.ts
│   │   │   └── yookassa/
│   │   ├── pickup-points/
│   │   │   ├── dto/
│   │   │   ├── entities/
│   │   │   ├── pickup-points.controller.ts
│   │   │   ├── pickup-points.module.ts
│   │   │   └── pickup-points.service.ts
│   │   └── notifications/
│   │       ├── notifications.gateway.ts
│   │       └── notifications.module.ts
│   ├── common/
│   │   ├── filters/
│   │   ├── guards/
│   │   ├── interceptors/
│   │   └── pipes/
│   ├── config/
│   │   └── database.config.ts
│   └── main.ts
├── prisma/
│   ├── schema.prisma
│   └── migrations/
├── test/
├── package.json
└── tsconfig.json
```

Рисунок 21 - Пример структура каталогов проекта бэкенда

Структура каталогов проекта фронтенде:

```
frontend/  
├─ lib/  
│   ├─ main.dart  
│   ├─ app.dart  
│   ├─ core/  
│   │   ├─ constants/  
│   │   ├─ routes/  
│   │   ├─ theme/  
│   │   └─ utils/  
│   ├─ data/  
│   │   ├─ models/  
│   │   ├─ repositories/  
│   │   └─ services/  
│   │       ├─ api_service.dart  
│   │       ├─ payment_service.dart  
│   │       └─ websocket_service.dart  
│   ├─ presentation/  
│   │   ├─ blocs/  
│   │   │   ├─ checkout/  
│   │   │   └─ order/  
│   │   └─ screens/  
│   │       ├─ cart_screen.dart  
│   │       ├─ pickup_point_selection_screen.dart  
│   │       ├─ time_selection_screen.dart  
│   │       ├─ checkout_screen.dart  
│   │       └─ order_status_screen.dart  
│   └─ widgets/  
│       ├─ custom_button.dart  
│       ├─ loading_indicator.dart  
│       └─ pickup_point_card.dart  
├─ assets/  
├─ pubspec.yaml  
└─ ...
```

Рисунок 22 - Пример структура каталогов проекта фронтенде

Заключение

Производственная практика способствовала закреплению и углублению полученных теоретических знаний, приобретению практических навыков и компетенций, подготовке к профессиональной деятельности путём самостоятельного решения задач, сформулированных в рамках индивидуального задания.

В результате выполненного исследования показано, что модель «Купи и заberi» (Click & Collect) является эффективным решением для ресторанного бизнеса, обеспечивающим снижение логистических затрат, увеличение трафика физических точек продаж и повышение удовлетворенности клиентов за счет гибкости выбора времени получения и отсутствия затрат на доставку.

Разработанная архитектура «клиент-сервер» с модульным монолитным API на NestJS в сочетании с кроссплатформенным клиентом на Flutter доказала свою состоятельность для проектов среднего масштаба, обеспечивая простоту внедрения и развертывания без ущерба для организации кода.

Интеграция платежного шлюза YooKassa с использованием механизма платежных токенов позволила обеспечить соответствие стандарту PCI DSS, минимизировать трение для пользователя и избежать хранения чувствительных платежных данных в бэкенде приложения.

Разработанная архитектура «клиент-сервер» с модульным монолитным API на NestJS в сочетании с кроссплатформенным клиентом на Flutter доказала свою состоятельность для проектов среднего масштаба, обеспечивая простоту внедрения и развертывания без ущерба для организации кода.

Интеграция платежного шлюза YooKassa с использованием механизма платежных токенов позволила обеспечить соответствие стандарту PCI DSS, минимизировать трение для пользователя и избежать хранения чувствительных платежных данных в бэкенде приложения.

К достоинствам/недостаткам реализованного программного продукта / полученной математической модели можно отнести:

Достоинства реализованного программного продукта

Таблица 9 - Достоинства реализованного программного продукта

№	Достоинство	Описание
1	Кроссплатформенность	Flutter обеспечивает единую кодовую базу для iOS и Android, сокращая время и затраты на разработку
2	Безопасность платежей	Использование платежных токенов исключает обработку и хранение данных карт на сервере
3	Модульность архитектуры	Четкое разделение на слои (Controller Service Repository) упрощает поддержку и масштабирование
4	Высокая производительность	Кэширование в Redis (TTL 5 минут - 1 час) снижает нагрузку на базу данных и ускоряет ответ API
5	Real-time уведомления	WebSocket обеспечивает мгновенное информирование пользователя об изменении статуса заказа
6	TypeScript-безопасность	Использование Prisma ORM гарантирует корректность запросов к БД на этапе компиляции
7	ACID-транзакции MySQL	обеспечивает целостность данных при создании заказов и обработке платежей
8	Географический поиск	Индексация координат точек выдачи позволяет быстро находить ближайшие локации

Недостатки реализованного программного продукта

Таблица 10 - недостатки реализованного программного продукта

№	Недостаток	Описание
1	Монолитная архитектура	При росте нагрузки потребуются горизонтальное масштабирование с балансировкой; отсутствует изоляция сервисов
2	Отсутствие офлайн-режима	Мобильное приложение требует постоянного подключения к интернету для работы
3	Зависимость от YooKassa	Смена платежного провайдера потребует существенной доработки модуля оплаты
4	Ограниченная аналитика	Отсутствует встроенный модуль расширенной аналитики поведения пользователей
5	Нет поддержки push-уведомлений	Реализованы только WebSocket-уведомления; для фоновых сценариев требуется интеграция с FCM/APNs
6	Отсутствие административной панели	Управление точками выдачи и заказами требует прямого доступа к базе данных или API
7	Нет автоматического резервирования	При отказе Redis теряются кэшированные данные и сессии пользователей
8	Ограниченная кастомизация времени работы	Не реализована поддержка особых графиков (праздничные дни, сезонные изменения)

Библиографический список

1. Flutter Documentation. [Electronic resource] – Google LLC, 2025 URL: <https://flutter.dev/docs> (Accessed: 20.11.2025).
2. Dart Language Tour. [Electronic resource]. Google LLC, Dart SDK URL: <https://dart.dev/guides/language/language-tour> (Accessed: 20.11.2025).
3. NestJS Documentation. [Electronic resource]. Kamil Mysliwiec, NestJS, MIT Licence 2017-2025 URL: <https://docs.nestjs.com> (Accessed: 21.11.2025).
4. Prisma Documentation. [Electronic resource]. Prisma Data, Inc Prisma Apache Licence 2.0, URL: <https://www.prisma.io/docs> (Accessed: 22.11.2025).
5. MySQL 8.0 Documentation. [Electronic resource]. ORACLE CORPORATION , MySQL. GPLv2. URL: <https://dev.mysql.com/doc/refman/8.0/en/> (Accessed: 22.11.2025).
6. Redis Documentation. [Electronic resource]. SALVATORE SANFILIPPO, PIETER NOORDHUIS. Redis. BD=SD 3-Clause License. Copyright 2006-2020, Salvatore Sanfilippo. URL: <https://redis.io/documentation> (Accessed: 23.11.2025).
7. WebSocket API Specification. [Electronic resource]. IETF. The WebSocket Protocol. RFC 6455. I. Fette; A. Melnikov 2011 URL: <https://html.spec.whatwg.org/multipage/web-sockets.html> (Accessed: 23.11.2025).
8. YooKassa API Documentation. [Electronic resource]. Group Tinkoff , SDK 2025 URL: <https://yookassa.ru/developers/api> (Accessed: 10.12.2025).
9. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
10. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
11. Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
12. Freeman, A. (2022). *Pro Flutter: Building High-Performance Cross-Platform Apps*. Apress.

Приложение



